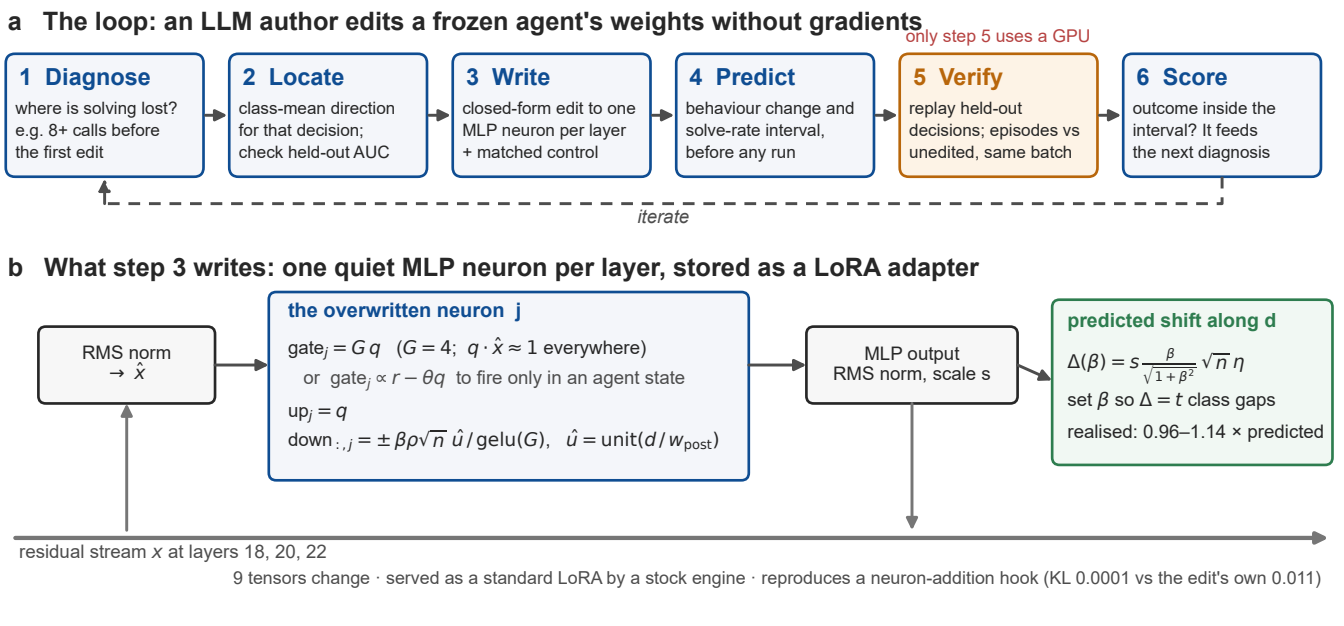

a The loop: an LLM author edits a frozen agent's weights without gradients
only step 5 uses a GPU
1 Diagnose
where is solving lost? e.g. 8+ calls before the first edit
2 Locate
class-mean direction for that decision; check held-out AUC
3 Write
closed-form edit to one MLP neuron per layer + matched control
4 Predict
behaviour change and solve-rate interval, before any run
5 Verify
replay held-out decisions; episodes vs unedited, same batch
6 Score
outcome inside the interval? It feeds the next diagnosis
iterate
b What step 3 writes: one quiet MLP neuron per layer, stored as a LoRA adapter
RMS norm
→ x̂
the overwritten neuron j
gate_j = G q (G = 4; q · x̂ ≈ 1 everywhere)
or gate_j ∝ r − θq to fire only in an agent state
up_j = q
down_:,j = ± βρ√n û / gelu(G), û = unit(d / w_post)
MLP output
RMS norm, scale s
predicted shift along d
Δ(β) = s β/√(1+β²) √n η
set β so Δ = t class gaps
realised: 0.96–1.14 × predicted
residual stream x at layers 18, 20, 22
9 tensors change · served as a standard LoRA by a stock engine · reproduces a neuron-addition hook (KL 0.0001 vs the edit's own 0.011)